

MANUAL INTEGRAL

Manual de usuario y de desarrollador

Guia completa para instalar, ejecutar, interpretar, probar, modificar y mantener AgroSmart Monitor.

AgroSmart Monitor

Version documentada: 0.1.0. Aplicacion de consola Java 17, Maven, JUnit 5 y GitHub Actions.

Documento preparado el 2 de septiembre de 2026

Repositorio: github.com/AlexitoXD1/agrosmart-monitor

Indice funcional

Parte	Contenido
I. Usuario	Proposito, requisitos, instalacion, ejecucion, salidas, reportes y solucion de problemas.
II. Desarrollador	Arquitectura, clases, reglas, build, pruebas, pipeline, trazabilidad y extensiones.
III. Operacion	Diagnostico, seguridad, mantenimiento, cambios seguros y checklist de entrega.

1. Que es AgroSmart Monitor

Es una aplicacion didactica de consola que simula el monitoreo de una camara de frio agricola. No necesita sensor fisico. Cada ejecucion crea cinco lecturas aleatorias con temperatura y humedad, clasifica la temperatura y genera un archivo cuando el estado es ALERT.

Rango / condicion	Estado	Accion
Temperatura <= 8.0 C	NORMAL	Mostrar lectura; no crear reporte.
8.0 C < temperatura <= 10.0 C	WARNING	Mostrar advertencia; no crear reporte.
Temperatura > 10.0 C	ALERT	Mostrar alerta y crear reporte en reports/.

Importante

Los umbrales son didacticos y no constituyen una recomendacion sanitaria ni un sistema de control de produccion.

PARTE I - MANUAL DE USUARIO

2. Requisitos

- Java Development Kit 17 o posterior.
- Apache Maven 3.8 o posterior.
- Terminal con acceso a la carpeta del proyecto.
- Git solo si se desea clonar o actualizar el repositorio.

```
java -version
mvn -version
git --version
```

2.1 Obtener el proyecto

```
git clone https://github.com/AlexitoXD1/agrosmart-monitor.git
cd agrosmart-monitor
```

Si ya se tiene esta carpeta, no es necesario clonar: basta con abrir una terminal en la raiz, donde estan **pom.xml** y **README.md**.

3. Primera ejecucion

- 1 Ejecutar **mvn test** para validar la instalacion y la logica.
- 2 Ejecutar **mvn package** para compilar y producir el JAR.
- 3 Ejecutar **java -jar target/agrosmart-monitor.jar**.

```
mvn test
mvn package
java -jar target/agrosmart-monitor.jar
```

Resultado esperado

La terminal muestra cinco lecturas. Al finalizar aparece **=== Fin de la simulacion ===**. Si alguna lectura supera 10 C, tambien se indica la ruta de un reporte nuevo.

4. Como interpretar la salida

```
=== AgroSmart Monitor - monitoreo de camara de frio (simulado) ===  
Lectura 1/5 -> [2026-09-02 10:30:12] Temp: 6.4 C | Humedad: 61.2% | Estado: NORMAL  
Lectura 2/5 -> [2026-09-02 10:30:12] Temp: 11.3 C | Humedad: 58.9% | Estado: ALERT  
-> Reporte de alerta generado: reports/alerta_20260902_103012.txt  
=== Fin de la simulacion ===
```

Campo	Significado
Lectura 1/5	Posicion actual y total del ciclo.
Fecha y hora	Momento local en que se creo el objeto TemperatureReading.
Temp	Valor simulado en grados Celsius, mostrado con un decimal.
Humedad	Valor simulado en porcentaje, mostrado con un decimal.
Estado	Resultado de aplicar los umbrales de TemperatureClassifier.
Ruta de reporte	Archivo creado unicamente para ALERT.

5. Reportes de alerta

Los archivos se guardan en **reports/** con el formato **alerta_yyyyMMdd_HH:mm:ss.txt**. El directorio se crea automaticamente.

```
ALERTA AUTOMATICA - AgroSmart Monitor  
Estado: ALERT  
Lectura: [2026-09-02 10:30:12] Temp: 11.3 C | Humedad: 58.9%
```

Colision posible

El nombre solo tiene precision de segundos. Si se generan dos alertas dentro del mismo segundo, el segundo reporte puede sobrescribir al primero. Para un uso real conviene añadir milisegundos o un identificador unico.

6. Problemas frecuentes del usuario

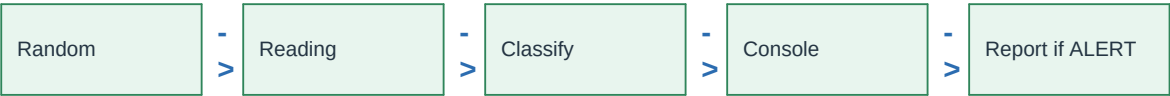
Sintoma	Causa probable	Solucion
mvn: command not found	Maven no esta instalado o no esta en PATH.	Instalar Maven y reabrir la terminal; comprobar mvn -version.
UnsupportedClassVersionError	El JAR fue compilado con una version de Java mas nueva.	Usar Java 17+ y confirmar java -version.
Unable to access jarfile	No se ejecuto package o se esta en otra carpeta.	Ejecutar mvn package desde la raiz.
No aparece reports/	No hubo temperatura > 10 C en esa ejecucion.	Repetir la simulacion; la generacion es aleatoria.
Pruebas fallan	Cambio incompatible o dependencia/cache dañada.	Leer el primer fallo; luego probar mvn -U clean test.
Caracteres raros	Terminal sin UTF-8.	Configurar la terminal en UTF-8.

7. Uso responsable

- No usar las lecturas simuladas para decisiones sobre alimentos o seguridad.
- No almacenar secretos, tokens ni contraseñas dentro del repositorio.
- No asumir que staging/ es un despliegue real; es una demostracion local del pipeline.
- Conservar reportes solo durante el tiempo necesario; pueden crecer con ejecuciones repetidas.

PARTE II - MANUAL DE DESARROLLADOR

8. Arquitectura y flujo de datos



El diseño separa generacion de datos, representacion, clasificacion y efectos de salida. La dependencia principal va desde **Main** hacia los componentes; **TemperatureClassifier** permanece como utilidad sin estado.

Clase	Contrato	Dependencias
Main	Orquesta cinco lecturas y espera 300 ms entre ellas.	SensorSimulator, classifier, report generator.
TemperatureReading	Dato inmutable de timestamp, temperatura y humedad.	LocalDateTime, DateTimeFormatter.
SensorSimulator	Entrega una lectura por llamada; permite semilla en tests.	java.util.Random.
TemperatureClassifier	Funcion pura double -> SensorStatus.	Sin dependencias externas.
SensorStatus	Enum NORMAL, WARNING, ALERT.	Ninguna.
AlertReportGenerator	Crea directorio y escribe el reporte de ALERT.	java.nio.file, LocalDateTime.

9. Detalle de las clases

9.1 TemperatureReading

El constructor recibe temperatura y humedad y captura **LocalDateTime.now()**. Los campos son finales. **toString()** produce la representacion usada tanto en consola como en el reporte.

9.2 SensorSimulator

El constructor sin argumentos usa una semilla no determinista. El constructor **SensorSimulator(long seed)** existe para pruebas repetibles. **nextReading()** usa intervalos continuos y redondea a dos decimales.

9.3 TemperatureClassifier

```
if (temperature <= 8.0) return NORMAL;
if (temperature <= 10.0) return WARNING;
return ALERT;
```

El orden de comparacion incluye exactamente 8.0 en NORMAL y 10.0 en WARNING. Cualquier cambio de umbral debe actualizar pruebas y documentacion en el mismo commit.

9.4 AlertReportGenerator

Crea **reports/** con **Files.createDirectories** y escribe con **Files.writeString**. Propaga **IOException**; Main tambien la declara. El estado se recibe como argumento, aunque Main solo llama al metodo para ALERT.

9.5 Main

La constante **READINGS_TO_SIMULATE** controla el tamaño del ciclo. El retraso de 300 ms mejora la lectura de la consola, pero hace que el metodo declare **InterruptedException**.

10. Compilacion y empaquetado

Comando	Uso	Salida
mvn clean	Elimina resultados previos.	Borra target/.
mvn test	Compila y ejecuta JUnit 5.	Reportes en target/surefire-reports/.
mvn package	Prueba y empaqueta.	target/agrosmart-monitor.jar.
mvn package -DskipTests	Empaqueta sin volver a ejecutar tests.	Usado en CI despues de mvn test.
java -jar target/agrosmart-monitor.jar	Ejecuta el manifiesto Main-Class.	Lecturas y posibles reports/.

10.1 Configuracion de pom.xml

- groupId: com.agrosmart; artifactId: agrosmart-monitor; version: 0.1.0.
- Compilacion source/target Java 17 y codificacion UTF-8.
- JUnit Jupiter 5.10.2 con alcance test.
- Surefire 3.2.5 para pruebas; maven-jar-plugin 3.4.1 para el manifiesto ejecutable.

11. Estrategia de pruebas

Suite	Cobertura actual	Riesgo protegido
TemperatureClassifierTest	NORMAL 6/8; WARNING 9/10; ALERT 10.1/15.	Errores en limites de la regla de negocio.
SensorSimulatorTest	20 lecturas con semilla 42 dentro de rangos.	Valores fuera de temperatura/humedad permitidas.

Pruebas adicionales recomendadas

Agregar casos NaN/infinito, formato de TemperatureReading, contenido y colisiones de reportes, permisos de escritura e integracion de Main con reloj y filesystem inyectables.

12. Pipeline GitHub Actions

```
push/main o pull_request/main
-> checkout@v4
-> setup-java@v4 (Temurin 17 + cache Maven)
-> mvn -B test
-> mvn -B package -DskipTests
-> copiar target/*.jar a staging/
-> upload-artifact@v4
```

La ejecución secuencial aplica fail-fast por defecto: si **mvn test** devuelve un código distinto de cero, los pasos normales posteriores se omiten. Los pasos internos Post pueden ejecutarse para limpiar recursos.

12.1 Como leer un fallo

- 1 Abrir Actions y seleccionar la ejecución asociada al commit.
- 2 Abrir el job build-test-deploy y localizar el primer paso rojo.
- 3 Leer la primera causa, no solo el último BUILD FAILURE.
- 4 Relacionar clase, método, línea, valor esperado y valor real.
- 5 Corregir localmente, ejecutar mvn test y subir un commit descriptivo.

12.2 Evidencia real del proyecto

Run	Commit	Estado	Resultado
#2	00f4204	Failure	Exit code 1; no hubo artefacto.
#3	9d77ab7	Success	JAR publicado, 6.34 KB, digest SHA-256 visible.

13. Guía para cambios seguros

Cambiar un umbral

- 1 Acordar la nueva regla y documentar el motivo.
- 2 Modificar TemperatureClassifier.
- 3 Actualizar casos de borde y nombres de pruebas.
- 4 Actualizar README y manuales si cambia el comportamiento externo.
- 5 Ejecutar mvn test y revisar el diff antes del commit.

Cambiar cantidad o rango simulado

Editar READINGS_TO_SIMULATE en Main o las formulas de SensorSimulator. Conservar pruebas de limites y ajustar el texto que describe los rangos. Para una solucion mantenible, mover estos valores a argumentos de linea de comandos o variables de entorno con validacion.

Conectar un sensor real

- Definir una interfaz ReadingSource con nextReading().
- Hacer que SensorSimulator la implemente.
- Crear un adaptador MQTT/HTTP separado con timeouts y reintentos.
- Inyectar la fuente en el orquestador y mantener tests con una fuente falsa.
- Validar datos, timestamps, unidades y comportamiento ante desconexion.

14. Seguridad y operacion

- Fijar permisos minimos del GITHUB_TOKEN y no imprimir secretos en logs.
- Revisar actualizaciones de actions/checkout, setup-java y upload-artifact.
- Agregar escaneo de dependencias y, si se usan contenedores, de imagenes.
- Conservar el digest del artefacto para trazabilidad e integridad.
- Separar ambientes y exigir aprobacion para un despliegue real a produccion.
- Definir retencion de reportes y artefactos para controlar datos y costos.

PARTE III - REFERENCIA RAPIDA

15. Checklist antes de entregar

- git status no contiene cambios accidentales ni archivos generados.
- mvn test termina con cero fallos y cero errores.
- mvn package crea target/agrosmart-monitor.jar.
- La aplicacion inicia con java -jar y finaliza el ciclo.
- README coincide con umbrales, rangos y comandos actuales.
- El workflow existe bajo .github/workflows/ y su YAML es valido.
- La ejecucion de Actions esta vinculada al commit correcto.
- El artefacto aparece solo despues de una ejecucion exitosa.

16. Comandos esenciales

```
# Estado e historial
git status --short
git log --oneline --graph -10

# Validacion local
mvn clean test
mvn package
java -jar target/agrosmart-monitor.jar

# Publicacion normal
git add <archivos>
git commit -m "Describir el cambio y su objetivo"
git push origin main
```

17. Enlaces

Repositorio: github.com/AlexitoXD1/agrosmart-monitor

Pipeline: github.com/AlexitoXD1/agrosmart-monitor/actions

Workflow: [.github/workflows/ci.yml](https://github.com/AlexitoXD1/agrosmart-monitor/blob/main/.github/workflows/ci.yml)

Configuracion Maven: [pom.xml](#)

Ayuda inicial: [README.md](#).

Definicion de terminado

Un cambio esta listo cuando la regla y sus pruebas coinciden, el build es reproducible, la documentacion refleja el comportamiento y el pipeline publica un artefacto trazable sin eludir la puerta de calidad.